Forgetting in World Models Does Not Follow Task Distance:
A Component-Level Benchmark and Two Negative Results

Jesús Pérez Bazarot
Universidad de Sevilla

August 14, 2026

Abstract

World models learn the dynamics of an environment so an agent can plan inside them. When such a model is trained on one environment and then another, the transition component that carries latent state forward is overwritten, and no benchmark measures that degradation at the level of the component itself: existing continual learning suites evaluate policies, or world models as integrated systems. We build one — three environment families, three levels of dynamic distance each, five continual learning methods, ten seeds in every cell that discriminates between them, 375 runs with 75 from-scratch reference pairs — and report what it says about the way this kind of experiment is usually designed and read.

Two results, both negative, and neither about which method wins. First, the distance axis we built the benchmark around does not order forgetting. Forgetting peaks at the *medium* level in all three families, so the labelled order is wrong in every one of them, and across the nine cells the label carries no rank information at all ($\rho = 0.00$, against 0.58 and 0.43 for the two quantities we measure). The cleanest case needs no cross-family comparison: one family's maximum level is its medium perturbation plus two more, applied to the same task pair, and it forgets *less*. Rerunning that pair at twice the training budget preserves the ordering and refutes the obvious explanation — the harder-perturbed task is *easier* to fit, not harder, because a cheetah at triple mass barely moves. Forgetting tracked how much the new task demanded of the model, not how far it sat from the old one.

Second, the forgetting happens almost entirely in the encoder, where component-level metrics — including ours — cannot see it. Fine-tuning's held-out reconstruction of the first task degrades by a factor of 811 while its prediction fidelity reads -5.58, i.e. improvement. EWC makes the dissociation exact and predictable: its Fisher information is identically zero on every encoder parameter, so it preserves the transition component almost perfectly (-0.03) with an encoder degraded by 800. A benchmark reporting only the isolated metric would certify that these models did not forget.

We characterise five methods rather than ranking them, our own included, and recommend that continual learning suites report a measured distance for each task pair and hold it to account rather than treating level labels as an experimental factor — a standard we also apply to our own measured distance, which fails it. Code, configurations and all 375 result files are released.

1 Introduction

A world model is an agent's learned model of how its environment evolves. In the decomposition of Ha and Schmidhuber [4] it has an encoder that maps observations to latent states, a transition component M that carries those states forward under actions, and a controller trained inside the resulting imagination. An agent that meets a sequence of environments must keep updating M,

and each update risks overwriting what M knew about the environments before — catastrophic forgetting, at the component that every downstream use of the model depends on. A transition model that has forgotten an environment's dynamics generates incoherent rollouts for it no matter how good the policy is.

Ha and Schmidhuber [4] name this as an open problem and leave it to future work. Since then it has been studied at the level of the agent: Continual World [19] measures forgetting in manipulation policies, and Kessler et al. [9] evaluate DreamerV2 as an integrated system and report policy-level return, forgetting and transfer. Both measure what the agent does. Neither isolates M, and we are not aware of a benchmark that does. The distinction is not bookkeeping: a system-level result cannot say whether a mitigation preserved the dynamics or preserved a policy robust to their degradation, and those two ask for different fixes.

So we built the missing instrument — three environment families, three levels of dynamic distance each, five continual learning methods, ten seeds in every cell that discriminates between them, 375 runs plus 75 from-scratch reference pairs — and this paper is mostly about what it told us that we had not asked. Both findings are negative, and neither is about which method wins.

The distance axis does not order forgetting. The benchmark, like most in this literature, is built around an ordinal notion of how far apart two tasks are, with the expectation that forgetting grows along it. It does not. Forgetting peaks at the *medium* level in all three families, so the labelled order is wrong in every one of them. Across the nine cells the label carries no rank information at all ($\rho = 0.00$), while both quantities we measure — the distance between the environments' transition models and the difficulty of the new task — do better (0.58 and 0.43). We read the result as forgetting scaling with what the new task demands of the model rather than with its distance from the old one, on evidence set out in Section 4.3 that is firmer than either correlation.

The cleanest evidence needs no comparison across families. In Gymnasium the maximum level is the medium perturbation plus two further ones, applied to the same task pair; a strict superset of a perturbation cannot produce less forgetting if the axis orders anything monotone, and it does. We rechecked that cell at twice the training budget, because the obvious objection is that the model simply never fits the harder task. The ordering holds, and the objection turns out to be backwards: the more heavily perturbed task is *easier* to fit, since a cheetah at triple mass and 40% gravity barely moves and generates correspondingly little to learn. The strongest perturbation in that family produces the poorest task, and the least forgetting.

The forgetting is in the encoder, where the metrics do not look. Component-level metrics have to hold a representation fixed in order to attribute a change to the component, and ours do: prediction fidelity and rollout divergence score M in a latent basis encoded once. That is the isolation we wanted, and it has a cost we can now measure. Fine-tuning's held-out reconstruction of the first task degrades by a factor of 811 while its prediction fidelity reads -5.58 — in the frozen basis, the model *improved*. EWC makes the dissociation exact and predicts it from its own definition: its Fisher information is taken over $\log P(z' \mid z, a)$, which contains no encoder parameter, so it is identically zero there. It preserves the transition component almost perfectly (-0.03) while its encoder degrades by 800. A benchmark reporting only PF and RD would certify that neither model forgot anything.

Contributions.

1. **A negative result about how these experiments are designed.** Level labels do not index forgetting, in any of three families, under any aggregation we tried, at two training

budgets and at two seed counts. We recommend reporting a measured distance per task pair and holding it to account — a standard our own measured distance fails, which we report rather than omit.

2. **A negative result about how they are read.** Forgetting in a world model concentrates in the component that isolated metrics are blind to, and the most widely used regularizer cannot reach it for a structural reason that is checkable in advance. Component-level benchmarks owe their readers both scales.

3. **The benchmark that produced them.** A component-level suite for forgetting in M: three metrics (prediction fidelity, rollout divergence, forward transfer), a measured distance between environments, three families, and a protocol that is declared in configuration, recorded in every result file, and refused by the runner if two budgets would meet in one cell. Five methods are characterised, not ranked.

What this paper does not claim. It does not claim that our own method wins; it is one of the five, and the benchmark's clearest finding about it is a failure mode (Section 4.6). It does not report a single aggregate score, because the one we started from is a median 88.5% rollout divergence by weight. It does not report a policy-impact metric, which an earlier version of this work announced and never implemented. And it supersedes that version entirely: none of its five findings survive re-measurement, for five instrumentation defects we document in Section 5.7, including one that reverses. That history is part of the argument rather than an embarrassment attached to it — a benchmark's claim on anyone's attention is that its numbers can be checked, and these were.

2 Background and related work

2.1 World models and the RSSM

A world model is an agent's internal, learned model of environment dynamics. In the formulation of Ha and Schmidhuber [4] it comprises a visual encoder, a transition model M, and a controller trained inside the model's imagination. The recurrent state-space model [5] gives M the form used throughout this paper — a deterministic recurrent state carried by a GRU, and a stochastic latent sampled from it — and the Dreamer line of work [6, 7, 8] builds agents on top of it.

The relevance of M to continual learning is specific: a transition model that has forgotten a previous environment's dynamics produces incoherent imagined rollouts for that environment *regardless of how good the policy is*, and any controller subsequently trained in that imagination inherits the corruption. Ha and Schmidhuber [4] name catastrophic forgetting as an open problem for world models and leave it to future work. Later systems mitigate it incidentally: DreamerV3 [8] trains from large replay buffers, which mixes experience across whatever the agent has seen, but this is a property of the data pipeline rather than a treatment of the transition component, and it says nothing about what happens to M when the mixing is removed.

2.2 Catastrophic forgetting and the three families of mitigation

Catastrophic forgetting [12, 13, 2] follows from the distributed, overlapping nature of neural representations: gradient steps that reduce loss on a new task move shared parameters away from optima for previous ones. The stability–plasticity dilemma [18] states the tension that any mitigation has to price — what reduces forgetting tends to reduce the capacity to learn what comes next.

Mitigations fall into three families, and we evaluate one representative of each. *Regularization* methods add a penalty that discourages movement in parameters deemed important for previous tasks, weighted by an importance estimate: a diagonal Fisher in EWC [10], a path integral in synaptic intelligence [20]. *Architectural* methods allocate separate capacity per task, whether by growing it [14] or by carving it out of a fixed network [11]. *Replay* methods retain and re-sample data from previous tasks.

All three were developed and are predominantly evaluated in classification, or on end-to-end reinforcement learning agents. That matters here for a reason that is not merely about coverage: an importance estimate is defined relative to a loss, and therefore has support only where that loss has gradients. When such a method is applied to a world model, which loss it is attached to decides which components it can protect at all — a point Section 4.5 turns into a measurement rather than an argument.

2.3 What existing continual learning benchmarks measure

The classification suites — Split-MNIST, Permuted-MNIST, Split-CIFAR — measure forgetting in discriminative models with no temporal structure, and do not transfer to a generative model of dynamics.

Within reinforcement learning, Continual World [19] is the standard benchmark: sequences of robotic manipulation tasks with actor-critic agents, reporting forgetting, forward transfer and performance at the level of the *policy*. Closest to our setting, Kessler et al. [9] study world models in continual reinforcement learning directly, evaluating DreamerV2 as an integrated system and finding that reservoir sampling in the replay buffer is the most effective mitigation, again measured by policy-level return, forgetting and transfer.

Both measure the agent. Neither isolates M, and neither defines a metric for the quality of imagined dynamics as such. We are not aware of a benchmark that does, and that gap is what the suite in Section 3 is built to fill. The distinction is not bookkeeping: a system-level result cannot say whether a mitigation preserved the dynamics or merely preserved a policy that was robust to their degradation, and those two call for different fixes.

Component-level measurement also sharpens a comparison we can now make. Kessler et al. [9] report that L2 regularization on the full DreamerV2 system performs poorly. We find that EWC restricted to the transition component preserves it almost exactly (Section 4.5) while leaving everything else unprotected — and, because its Fisher is identically zero outside the transition loss, this is structural rather than a matter of tuning. Read together, the two results suggest that the disappointing behaviour of regularization in world models is less about the penalty than about the mismatch between where its importance signal has support and where the forgetting actually happens.

2.4 Task distance as an experimental factor

Continual learning results are routinely reported against a notion of how related the tasks in a sequence are, and benchmarks are built to span that notion: sequences are assembled from tasks judged similar or dissimilar, and the resulting ordering is then treated as an experimental factor against which forgetting is expected to grow. Our own design does exactly this, with three levels of dynamic distance per family (Section 3.4), and the previous version of this work described them as controlled.

What is usually missing — and was missing from our own design — is a *measured* quantity to which the ordering is answerable. Where the environments differ in explicit physical parameters,

differencing them is available and exact (Equation 5); where they differ in body or layout, there is nothing to difference, and the ordering rests on judgement. Section 4.2 reports what happened when we checked ours against both the forgetting it produced and a measured distance between the environments' transition models, and Section 5.1 states the recommendation that follows. The recommendation is aimed at this practice rather than at any particular benchmark.

2.5 Where forgetting is measured inside a model

A metric for a multi-component model has to hold something fixed in order to attribute a change to anything, and what it holds fixed bounds what it can detect. Metrics defined over a representation — ours included, since PF and RD score the transition component in a latent basis encoded once (Section 3.2) — are by construction unable to observe drift in the mapping that produced that basis. This is the price of component isolation, not a defect of a particular estimator, and the same trade appears wherever forgetting is attributed to a part of a network rather than to the whole.

The consequence is that a component-level benchmark owes its readers both scales. Reporting only the isolated metric can certify that a method preserved the component it was pointed at while the model as a whole became unusable for the old task. Section 4.5 is that situation, measured.

3 The benchmark

3.1 What is being measured, and in which component

A world model factorises into three learned parts: an encoder E that maps an observation o_t to a latent z_t, a transition component M that carries (z_t, a_t) forward in time, and a decoder D that maps latents back to pixels. Continual-learning results for model-based agents are almost always reported at the level of the agent — returns, or success rates, after a task switch — which confounds the three parts with each other and with the policy trained on top of them. This benchmark measures M.

We use the recurrent state-space model of Hafner et al. [5] as the common substrate, in the configuration of Hafner et al. [6], with a convolutional VAE encoder–decoder [4]. Observations are rendered to 64×64 RGB and scaled to $[0, 1]$ in all three families, so the same architecture spans them. The deterministic state is carried by a GRU cell,

$$h_t = \mathrm{GRU}\big([z_{t-1}, a_{t-1}], h_{t-1}\big), \qquad z_t \sim \mathcal{N}\big(\mu_\theta(h_t), \operatorname{diag} \sigma_\theta^2(h_t)\big), \tag{1}$$

and $M = (\mathrm{GRU}, \mu_\theta, \sigma_\theta)$ is the object the forgetting metrics score. The model is trained on reward-free rollouts from a random policy, so nothing in the pipeline depends on a task's reward function — a property that matters more than it sounds, and Section 3.4 returns to it.

A run trains one model on a sequence of tasks $T_1 \dots T_k$, and measures on T_1 what training on the rest cost. All experiments here use $k = 2$: we write task A and task B, and every metric is an A-versus-A comparison of the model before and after training on B. Each task contributes held-out episodes that are never trained on, and the evaluation sets are built from those.

3.2 Three metrics, and what each one can see

Write M_A for the model after task A and M_B for the same model after task B.

Prediction fidelity (PF). The evaluation set D_A is a set of latent transition triples (z, a, z') drawn from held-out task-A episodes. PF is the one-step negative log-likelihood the later model assigns to them, minus the earlier model's:

$$\mathrm{PF} = \mathrm{NLL}(M_B, D_A) - \mathrm{NLL}(M_A, D_A), \qquad \mathrm{NLL}(M, D) = -\,\mathbb{E}_{(z,a,z')\sim D}\big[\log P_\theta(z' \mid z, a)\big]. \tag{2}$$

Positive PF means the transition component got worse at task A. The target is z', the *next* latent: scoring against z would measure how far the transition moves the state rather than how well it predicts it.

Rollout divergence (RD). PF is a one-step quantity, and a transition model can be locally accurate while compounding error over a horizon. RD imagines forward from held-out task-A start states under both models, driven by the same actions, and averages the per-step divergence between their predicted state distributions:

$$\mathrm{RD} = \frac{1}{H}\sum_{t=1}^{H} \mathbb{E}\Big[D_{\mathrm{KL}}\big(P_A(z_t \mid h_t^A) \,\big\|\, P_B(z_t \mid h_t^B)\big)\Big]. \tag{3}$$

Each model propagates its own predicted mean, so the two rollouts are allowed to separate; the actions at each step are resampled from the evaluation set's own action distribution rather than from a Gaussian, which would be off-manifold for bounded continuous actions and meaningless for one-hot discrete ones. RD is a KL and is therefore unbounded above, which is a property of the metric that Section 4.1 has to handle rather than an artefact.

Forward transfer (FT). Forgetting is only half of a continual learning result. FT asks what having seen task A did for learning task B, against the counterfactual of a model that trained on B alone under the same budget and the same data:

$$\mathrm{FT} = \mathcal{L}_B^{\mathrm{px}}\big(\text{trained on B from scratch}\big) - \mathcal{L}_B^{\mathrm{px}}\big(\text{pretrained on A}\big), \tag{4}$$

where $\mathcal{L}_B^{\mathrm{px}}$ is held-out reconstruction error on task B in pixel space. Positive FT means knowledge of A helped. It is measured in pixels, not in latent NLL, and the reason is not cosmetic: the two models were trained independently, so their latent spaces are unrelated bases and a latent likelihood would score one model in coordinates belonging to the other. Pixels are the one scale the two arms share. One caveat travels with the number: replay's pretrained arm trains on A and B together during phase B, so it spends half its updates on task-A data, and its FT mixes transfer with a halved effective budget on B.

Declared scope. D_A is encoded once, by M_A, and both models are scored on those fixed latents. This is what isolates M: letting each model encode with its own drifting VAE would compare every model against its own moving coordinates rather than against a common evaluation set. The cost is that PF and RD cannot see drift in the encoder at all, and Section 4.5 shows that the blind spot is where most of the forgetting happens. We therefore report, for every run, the held-out task-A reconstruction error in pixels alongside PF and RD. A second and smaller restriction: the transitions in D_A are scored from a zero initial hidden state, so PF measures the one-step map and not the recurrent carry.

On aggregation, and on a metric we withdrew. An earlier version of this benchmark combined PF, RD and a policy impact score into a single scalar. We report PF and RD separately (Section 4.1 gives the measured reason), and we withdraw the policy impact score: scoring a policy trained inside the model's imagination requires a controller this benchmark does not build, and reporting an unimplemented term as a zero is worse than not announcing it. The suite is PF, RD and FT.

3.3 Two ways to measure the distance between two environments

The design varies the distance between task A and task B, so that distance has to be quantified rather than asserted. Two instruments are available and they are not interchangeable.

The *parametric* distance differences the physics vector $\phi = [\,\text{gravity}, \text{mass scale}, \text{friction scale}\,]$ directly,

$$d_{\text{param}} = \frac{\|\phi_A - \phi_B\|_2}{\|\phi_A\|_2}, \tag{5}$$

and is exact where it applies. It applies to four of our nine cells. It is undefined for a change of layout or of body: the pairs that swap cheetah for walker leave ϕ at its default in both tasks, so Equation 5 would score the largest construction change in that family as no change at all.

The *transition* distance is defined wherever transitions can be sampled. It trains one plain RSSM per environment, from scratch, and takes the expected divergence between their one-step predictions on shared held-out task-A transitions:

$$d_{\text{trans}} = \mathbb{E}_{(z,a)\sim D_A}\Big[D_{\text{KL}}\big(P_A(z' \mid z,a) \,\big\|\, P_B(z' \mid z,a)\big)\Big]. \tag{6}$$

It is a property of the task pair and the seed, not of the continual learning method, so it is computed once per cell and seed and all five methods read the same value back. The two reference models it needs are the same two that supply the from-scratch arm of Equation 4, which is why measuring both cost one pair of extra trainings per cell and seed rather than two: 75 pairs against the grid's 375 runs, a 20% surcharge.

Equation 6 carries a caveat we have not resolved, and it is the same one that keeps FT in pixel space. The two reference models are trained independently, so P_B is evaluated on latents produced by A's encoder — a basis it has never seen. Part of what d_{trans} measures is therefore basis mismatch rather than dynamics mismatch, and its absolute magnitudes are not comparable across families for that reason (Table 1 puts MiniGrid near 20 and DMControl near 5). We use it only to rank cells within a family and to rank the nine cells against each other, and Section 4.3 reports where that ranking holds and where it fails.

3.4 Environment families and distance levels

Three families cover three regimes an M has to cope with, and Table 1 lists the pairs. **MiniGrid** [1] is discrete: seven one-hot actions, and levels built by changing the layout, from a larger empty room to four rooms to a keyed corridor. **Gymnasium** is continuous control with variable physics: six actuators on MuJoCo's HalfCheetah [16], with the three levels applying progressively stronger perturbations to the same body. **DMControl** [17] is the visually hardest of the three, and its levels mix a change of body with the same physical perturbation Gymnasium's maximum level applies.

Gymnasium's three levels are nested by construction, and this turns out to carry more weight than the rest of the design: both the medium and maximum levels move gravity from 9.8 to 4.0, and the maximum additionally scales mass and friction. It is strictly the medium perturbation

Family	Level	Task A	Task B	Seeds	d_{param}	d_{trans}
MiniGrid	min	`Empty-5x5`	`Empty-8x8`	10	–	22.35
	med	`Empty-8x8`	`FourRooms`	10	–	20.64
	max	`FourRooms`	`KeyCorridorS3R1`	10	–	25.19
Gymnasium	min	`HalfCheetah` ($g = 9.8$)	`HalfCheetah` ($g = 7$)	5	0.283	28.75
	med	`HalfCheetah` ($g = 9.8$)	`HalfCheetah` ($g = 4$)	5	0.586	30.86
	max	`HalfCheetah` ($g = 9.8$)	`HalfCheetah` ($g = 4$, mass $\times 3$, friction $\times 0.5$)	10	0.622	24.08
DMControl	min	`cheetah/run` ($g = 9.81$)	`cheetah/run` ($g = 7$)	5	0.284	1.87
	med	`cheetah/run`	`walker/run`	10	–	7.58
	max	`cheetah/run`	`walker/run` ($g = 4$, mass $\times 3$, friction $\times 0.5$)	10	–	7.46

Table 1: The nine cells, read from the stored results rather than from the configuration files. d_{param} is left empty where differencing a physics vector would misdescribe the pair (Section 3.3); d_{trans} is the median over the cell's seeds and is comparable within a family, not across families.

plus more perturbation, applied to the same task pair. Any claim that the axis orders something monotone can be checked against those two cells alone, with no appeal to how families compare.

Two constraints shaped the levels and are worth stating because they are easy to trip over. One world model spans both tasks of a pair, so its GRU has a single action width and the two tasks must have the same number of actuators — which in dm_control leaves only cheetah and walker. And because the model is trained on reward-free rollouts, two dm_control tasks from the same domain are the *same environment* as far as this benchmark can see: they share physics and initial-state distribution and differ only in a reward function nothing here consumes. A level built as `walker/run` against `walker/stand` produces a bit-identical copy of its neighbour, which is how we built one before we caught it.

3.5 Methods evaluated

Five methods, spanning the three standard families of mitigation plus two bounds.

- **Fine-tuning** — sequential training with no protection. The lower bound.
- **Infinite replay** — an unbounded buffer of every past task, sampled uniformly during phase B. The approximate upper bound, and the only method here that sees task-A data while learning task B.
- **EWC** [10] — a diagonal-Fisher quadratic penalty around the post-task-A weights, $\lambda = 1000$. The Fisher is estimated from per-sample gradients of $\log P_\theta(z' \mid z, a)$.
- **Progressive networks** [14] — one GRU column per task, previous columns frozen, lateral projections from the last frozen column into the new one. Like every method here it is evaluated without a task oracle: inference runs through the newest column, since no task label is available at test time to route by. This differs from the original protocol, where the label selects the column — under oracle routing the frozen task-A column would score PF = 0 by construction — and the numbers of Section 4.6 are statements about the task-agnostic reading.
- **UG-MTM** — our own method: the deterministic transition is replaced by a pool of four GRU experts [15] gated by an MC-dropout estimate of epistemic uncertainty [3]. After task A the VAE and the μ/σ projection head are frozen along with the previous experts, so only the

Parameter	Value
Episodes collected per task (random policy)	20
Gradient updates per task	5000
Batch size	8
Sequence length	5
Learning rate (Adam)	1e-03
Latent dimension	32
GRU hidden dimension	512
KL weight β	1.0
Held-out task-A episodes	10
Transitions in D_A	100
Held-out frames for pixel reconstruction	200
RD rollout horizon	15
RD start states	50
Transitions in EWC's Fisher estimate	50
EWC λ	1000
UG-MTM MC-dropout samples (training)	3
Seeds per cell	5–10

Table 2: The executed protocol. Generated from the `protocol` block that every result file stores, not transcribed from the configuration.

newly activated expert, the uncertainty head and the threshold network train on task B. It is included as one of five methods, not as the paper's result.

Two properties of EWC's penalty follow from its definition and are worth naming before the results rather than after. Its Fisher is defined over $\log P_\theta(z' \mid z, a)$, which contains no encoder parameter, so it is *exactly* zero on all twenty VAE tensors: the penalty cannot constrain the encoder even in principle. It is likewise zero on the GRU's hidden-to-hidden weights, because the transitions the Fisher is estimated from all start from $h = 0$. Both zeros are pinned by tests, and Section 4.5 measures what the first one costs.

3.6 Protocol

Table 2 gives the budget. Per cell and seed: collect episodes from task A under a random policy, train, snapshot, collect from task B, train again, and evaluate. Rollout collection is shared across the five methods and the reference pair of that cell and seed, so all six see identical data.

The protocol is declared in the configuration, and every result file records the block it was produced under. The runner reads all of it and hardcodes none of it; it skips cells it has already computed, and *refuses* cells produced under a different protocol or on a different task pair, so two budgets can never be averaged into one table cell. Table 2 is generated from those stored blocks rather than from the configuration for the same reason: a protocol table transcribed by hand describes an intention, and what a benchmark owes its readers is a description of what ran.

Runs are reproducible from their seed. Seeding covers the global RNGs, cuDNN, and — the part that is easy to miss — each environment's own generator, including Gymnasium's action space and dm_control's `RandomState`; without those the five seeds of a cell train on five different datasets and are not replicates. Instrumentation is wrapped so that measuring cannot change a result, which is

load-bearing here because UG-MTM's gate keeps MC-dropout active at evaluation time by design and so consumes the random stream. We reproduced a cell bit for bit eight days after it first ran.

Each family runs in its own process. This is not tidiness: dm_control cannot obtain an OpenGL context in a process where MuJoCo already holds one, and a 30-hour run died at exactly that boundary before the runner was split.

4 Results

We ran the full grid: three environment families × three dynamic-distance levels × five methods, each training a world model on task A and then on task B under one protocol (5000 gradient steps, 20 random-policy episodes per task, batch 8, sequence length 5). Every cell ran five seeds, and the six cells that discriminate between methods — Section 4.4 identifies the other three as controls — were then extended to ten, for 375 runs in total. A further 75 reference pairs — one model per environment, trained from scratch — supply the from-scratch arm of forward transfer and the measured distance d_{trans}. No run dropped a NaN step.

4.1 How the numbers are reported

Cells are summarised by the *median* over their seeds, with the full seed range where space allows. This is not cosmetic. RD is a KL divergence and is unbounded above, so a method whose post-task-B model becomes overconfident acquires a heavy right tail by construction: our own method, UG-MTM on MiniGrid at maximum distance, produces 17.7, 40.0, 70.8, 520, 575, 577, 1490, 4364, 4963 and 8135 across its ten seeds, and the mean of those, 2075, is larger than seven of them. The skew is common rather than exceptional — 17 of the 180 forgetting cells have an upper half that stretches more than five times further from the median than their lower half — so we report medians throughout and mark the skewed cells ($*$) rather than averaging the tail away.

PF and RD are reported separately. The aggregate WMF of the earlier formulation is dominated by RD, which supplies a median of 88.5% of it and at least three quarters of it in 37 of the 45 cells, so a single scalar would be RD with extra steps. The exceptions are informative rather than reassuring: they are the cells where RD is itself near zero — DMControl's control cell, and every DMControl cell of the method that freezes its encoder. Comparisons between methods are paired by seed and tested with an exact two-sided permutation test over all 2^n sign flips, alongside the paired effect size d_z and the win count. We do not report parametric p-values. The seed count matters for what those tests can say and is why we bought more of them: at $n = 5$ the exact test cannot return a p below $2/2^5 = 0.0625$, so no five-seed comparison can reach conventional significance at all, whereas at $n = 10$ the floor is $2/2^{10} \approx 0.002$. Comparisons drawn from the three control cells remain at five seeds and inherit that ceiling; the ones the paper rests on do not.

4.2 The labelled distance axis does not order forgetting

The benchmark was built around an ordinal axis: each family has a minimum, a medium and a maximum dynamic distance, and the expectation encoded in that design is that forgetting grows along it. It does not.

Figure 1 and Table 3 give RD per family and level. In all three families the maximum sits at the *medium* level, and the labelled order min $<$ med $<$ max is therefore wrong in every one of them. Three out of three is the most robust part of this result: it does not depend on how cells are aggregated, on which of the two forgetting metrics is read, or on any threshold.

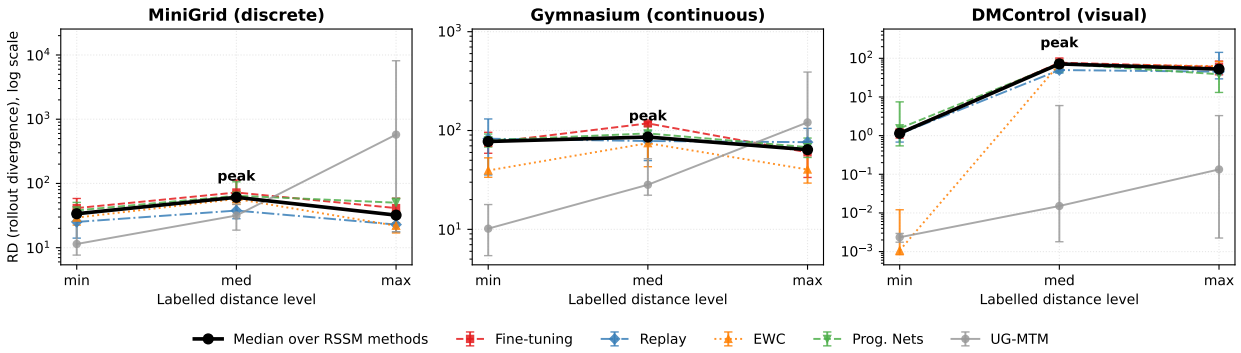

Figure 1: Forgetting against the labelled distance level, one panel per family, log scale. Coloured lines are the five methods (median over seeds, bars to the seed extremes); the black line is the median over the four methods that share the RSSM architecture, which is what the peak is stated over. **The peak is at the medium level in all three families**, so the labelled order is wrong in every one of them. UG-MTM is drawn in grey and excluded from the aggregate: it freezes its encoder, which puts it orders of magnitude below the rest in DMControl.

Family	min	med	max	Peak
MiniGrid	33.73	**60.88**	32.11	med
Gymnasium	77.45	**85.91**	64.01	med
DMControl	1.15	**71.64**	52.61	med

Table 3: The aggregate of Figure 1 as numbers: rollout divergence by family and labelled distance level, over the four methods that share the RSSM architecture (median over methods of each method's median over seeds).

Gymnasium makes the point without any appeal to how the levels compare across families, because there the two upper levels are nested by construction. Both move HalfCheetah from gravity 9.8 to 4.0; the maximum level additionally scales mass by 3 and friction by 0.5. It is strictly the medium perturbation plus more perturbation, and it produces *less* forgetting (64.01 against 85.91).

The obvious objection is that this is a statement about our budget rather than about forgetting: if at the maximum level the model never fits task B, nothing gets overwritten and the cell reads low for an uninteresting reason. We tested it by rerunning both cells at twice the training budget, and it does not hold up. The ordering survives — restricted to fine-tuning and to the five seeds both budgets share, so that the two are compared on the same estimator, the cells read 118.04 (medium) against 58.79 (maximum) at 5000 updates and 144.70 against 102.18 at 10000 — and the mechanism the objection proposes is the opposite of what happens. Task B is *easier* to fit at the maximum level than at the medium one, at both budgets (17.44 against 27.28, and 15.14 against 24.65): the heavier, lower-friction, low-gravity cheetah moves less, and a less varied task is a task with less to learn from. What we report as robust is the direction, which survives the doubling; the magnitude is budget-dependent, the ratio between the two cells falling from 2.01 to 1.42.

The aggregate in Table 3 deliberately excludes UG-MTM. It freezes its encoder, which puts its RD three orders of magnitude below the other four methods in DMControl (0.02 against 50 to 77 at the medium level); pooling all five would average two scales rather than read the axis. Its own numbers appear in Section 4.6 and in Table 7.

Family	Level	RD	d_{trans}	Task-B difficulty
MiniGrid	min	33.73	22.35	2.01
	med	60.88	20.64	42.60
	max	32.11	25.19	23.59
Gymnasium	min	77.45	28.75	24.63
	med	85.91	30.86	27.28
	max	64.01	24.08	17.44
DMControl	min	1.15	1.87	15.43
	med	71.64	7.58	34.07
	max	52.61	7.46	40.54
Spearman ρ with RD (predictor)		0.00 *labelled level*	0.58 *measured*	0.43 *measured*

Table 4: The nine cells behind the rank correlations, and the correlations themselves. RD and task-B difficulty are aggregated as in Table 3; d_{trans} is a property of the task pair and is the median over seeds.

4.3 What does predict forgetting

If the labelled level does not order forgetting, something else may. We compare three candidate predictors across the nine cells: the labelled level itself, the measured distance d_{trans} between the two environments' transition models, and the difficulty of task B — operationalised as the held-out reconstruction error a model reaches on task B after training on it, i.e. how hard task B is to fit at this budget.

Table 4 shows the outcome. The labelled level carries no information at all ($\rho = 0.00$). Both measured quantities do better: the transition distance reaches $\rho = 0.58$ and task-B difficulty $\rho = 0.43$. In MiniGrid and Gymnasium the difficulty column rises and falls with RD, and the medium level is exactly where task B is hardest to fit (42.60 against 2.01 and 23.59 in MiniGrid; 27.28 against 24.63 and 17.44 in Gymnasium).

The reading we draw is that **forgetting scales with how much the new task demands of the model, not with how far it is from the old one**, and we state it as the best available account rather than a demonstrated mechanism, for two reasons. DMControl does not follow it: task-B difficulty there keeps rising from minimum to maximum (15.43, 34.07, 40.54) while RD falls after the medium level. And the ranking between the two measured predictors is not stable — before we extended the discriminating cells to ten seeds, task-B difficulty led at 0.58 with the transition distance at 0.53, and adding seeds reversed them. What does not move is the negative result: the labelled level was uninformative at five seeds and is uninformative at ten.

That instability is the first caveat, and it is worth reading literally rather than as boilerplate. These are rank correlations over $n = 9$ points drawn from three families whose scales differ by an order of magnitude, and we have now watched the ordering of the top two change under a manipulation that left the finding itself untouched. What the correlations establish is that both measured quantities beat the label decisively and that neither is resolved against the other; the load-bearing evidence for the account is not here but in Section 4.2, where the Gymnasium pair differs by a construction we control and the more heavily perturbed level is measurably the easier task. The repeated peak is the robust half of the finding, and the correlations are its interpretation. Second, and more seriously, d_{trans} is not a drop-in replacement for the labelled axis, and its $\rho = 0.58$

— now the highest of the three — should not be read as one. Between families it separates cleanly, DMControl's cells sitting near 7 where MiniGrid's sit near 22, and that separation is what most of the correlation is made of. *Within* a family it mostly does not separate at all. Gymnasium's three levels have medians of 28.75, 30.86 and 24.08, spanning about seven units, while the individual cells span fifteen to twenty-three across seeds ($[21.5, 37.0]$, $[18.5, 37.2]$, $[17.7, 40.6]$): the rank order is an order over medians that sit inside each other's noise. MiniGrid inverts min and med on the same terms (22.35 against 20.64), and DMControl's medium and maximum are indistinguishable (7.58 against 7.46, each spanning about nine). The budget rerun of Section 4.2 makes the diagnosis directly: at twice the updates, Gymnasium's medium and maximum medians swap places (57.30 against 71.51) while their seed ranges remain all but identical. So the correlation is carried by the between-family comparison, which is the one d_{trans} is least entitled to make (Section 3.3), and it leads the table for a reason that does not license using it as an axis.

What survives is narrower than a replacement axis and still worth stating: a measured distance is answerable to evidence in a way a label is not, and this one answered. It says its own within-family orderings are not resolved at five seeds, and it changes when the budget changes — which is itself a finding about Equation 6, since a distance between two environments should not depend on how long one trains the models used to measure it.

4.4 Three cells produce no forgetting, and we report them as controls

Not every cell has something to forget. We call a cell a *control* when no method's held-out task-A reconstruction error grows by more than 10% after training on task B. The threshold is not a knife edge: in every cell that does produce forgetting, the worst method ends between 14.8× and 848× its original error, so the nearest one sits more than 13× above the threshold and any cutoff between 1.1 and 14 selects the same three cells.

Three of the nine qualify (marked † in Table 5): Gymnasium at minimum and medium distance, and DMControl at minimum. In the first two the task pair is the same HalfCheetah under different gravities, so training on task B keeps teaching the model to reconstruct task A. This is what the bottom of a distance axis is supposed to look like, and the benchmark detects it; it also means those cells discriminate between methods on nothing, and the effective grid is six cells, not nine.

The criterion is deliberately blind to RD, and Gymnasium's medium cell shows why: it loses nothing in pixels and simultaneously carries the highest RD in its family (85.91). A cell can leave the encoder intact and still move the transition component a long way. That is the subject of the next section.

4.5 Forgetting happens in the encoder, where the metrics cannot see it

PF and RD are computed on latents encoded once, by the post-task-A model, which is what isolates the transition component M — the benchmark's declared scope. The cost of that isolation is that both metrics are blind to drift in the encoder itself, and Table 5 shows how large the blind spot is.

Fine-tuning's held-out task-A reconstruction grows by a factor of 811 in MiniGrid at minimum distance, and its PF for that cell is −5.58: measured in the frozen latent basis, the model *improved*. This is not an isolated sign error. Fine-tuning's PF is negative in seven of the nine cells (Table 6) while its encoder is destroyed in six of them.

EWC makes the dissociation sharpest, and it was predicted before it was measured. Its Fisher information is defined over $\log P(z' \mid z, a)$, so it is *exactly* zero on every encoder parameter: the penalty cannot constrain the VAE even in principle. Both halves of that prediction hold. In MiniGrid at minimum distance EWC preserves the transition component almost perfectly (PF = −0.03,

Family	Level	Fine-tuning	Replay	EWC	Prog. Nets	UG-MTM
MiniGrid	min	811	1.31	800	797	1.00
	med	829	1.18	848	825	1.00
	max	32	1.02	31	31	1.00
Gymnasium	min†	0.94	0.90	0.94	0.97	1.00
	med†	1.04	0.92	1.05	1.06	1.00
	max	14	1.04	15	14	1.00
DMControl	min†	1.02	1.00	1.02	1.02	1.00
	med	16	0.37	16	16	1.00
	max	17	0.51	17	16	1.00

Table 5: Held-out task-A reconstruction error after training on task B, as a factor of the same model's error before it. 1.00 means nothing was lost, 811 means the error grew by a factor of 811. † marks control cells (Section 4.4).

Family	Level	Fine-tuning	Replay	EWC	Prog. Nets	UG-MTM
MiniGrid	min	-5.58	-5.98	-0.03	4.06	3.25
	med	-4.23	-7.23	0.04	6.10	11.71
	max	0.60	-6.32	0.07	14.43	38.65
Gymnasium	min	-8.27	-8.00	0.02	-0.03	-0.03
	med	-8.40	-8.27	-0.01	-0.36*	-0.05
	max	0.52	-6.00	0.04	15.09	15.66
DMControl	min	-5.43	-5.23	0.01	5.80	0.11
	med	-18.02	-13.54	-11.11*	-3.05	0.47
	max	-16.95	-13.39	-9.56	-3.53	2.08

Table 6: PF (prediction fidelity) by method and cell; median over the cell's seeds (ten in the six discriminating cells, five in the controls). Positive means the transition component got worse at task A. $*$ marks cells whose seed sample is right-skewed.

against -5.58 for fine-tuning) while its encoder degrades by a factor of 800 — indistinguishable from fine-tuning's 811. Across the grid its PF stays within $|0.07|$ in seven of the nine cells.

Replay is the only method that keeps both. Its task-A reconstruction never grows by more than 31%, and in DMControl it *improves* (factors of 0.37 and 0.51), which is what training on A and B together should do.

We report this as a scope declaration rather than a defect: measuring M in a fixed latent basis is the stated purpose, and the alternative — letting each model encode with its own drifting VAE — would compare each model against its own moving coordinates rather than against a common evaluation set. But a benchmark that reports only PF and RD would report that fine-tuning does not forget, and that is why every run in this suite records the pixel scale alongside them.

4.6 What the benchmark says about the methods

Replay is not insufficient here. At ten seeds the six forgetting cells separate into three groups rather than a trend. In four of them — all three MiniGrid levels and DMControl's medium —

Family	Level	Fine-tuning	Replay	EWC	Prog. Nets	UG-MTM
MiniGrid	min	41.36	25.18	29.39	38.07	11.42*
	med	72.29	37.82	57.34	64.43*	31.65
	max	41.33	22.89	21.95	50.11	576*
Gymnasium	min	75.30	82.62	39.42	79.59	10.17
	med	118	78.32	74.45	93.50	28.26
	max	60.18	76.23	40.32	67.84	121
DMControl	min	1.19	1.11	0.00*	1.54*	0.00
	med	77.20	49.70	70.75	72.53	0.02*
	max	60.13	45.09*	62.87	38.68	0.13*

Table 7: RD (rollout divergence) by method and cell; median over the cell's seeds. $*$ marks right-skewed cells.

infinite replay has lower RD than fine-tuning in *every* seed, at the exact test's floor of $p = 0.002$, with d_z from -2.28 to -3.11. In one, Gymnasium at maximum distance, replay is *worse*, in nine seeds of ten (76.23 against 60.18; $p = 0.006$, $d_z = +1.14$). In the last, DMControl's maximum, there is no effect to report ($p = 0.68$, $d_z = -0.14$): the medians differ but the seeds do not agree on a direction, and at five seeds we would have called that a win for replay.

The direction in the four is the one expected a priori, since replay trains on A and B together. We report it because an earlier version of this benchmark reported the opposite at $p < 0.001$ with five seeds, and that reversal is attributable to defects in the measurement rather than to the setting.

EWC protects the transition component and only that. Discussed in Section 4.5. Its near-zero PF is not universal: in DMControl's two forgetting cells it reads -11.11 and -9.56, in line with the other methods, so the protection is not uniform across families either.

Progressive networks are the only baseline with consistently positive PF in MiniGrid (4.06, 6.10, 14.43), i.e. the benchmark scores them as forgetting more of the transition component, not less — while their encoder degrades like fine-tuning's. The number reads on the task-agnostic protocol declared in Section 3.5: evaluation routes through the active column because no task label is available at test time, whereas oracle routing through the frozen task-A column would put PF at zero by construction. Under task-agnostic inference, column capacity does not help a component that is not the one being overwritten.

UG-MTM does not forget because it does not learn. Its encoder factor is exactly 1.00 in all nine cells — it is frozen, so task-A reconstruction is preserved bit for bit — and it pays for that in forward transfer: FT of -557, -526 and -1642 in MiniGrid (Table 8), where no other method leaves the range $[-8.52, +2.01]$ anywhere in the grid. The cost nearly vanishes when the two tasks look alike (-4.33 in Gymnasium at minimum distance), which locates the regime where structural isolation is affordable rather than establishing that it works.

Freezing the encoder produces overconfident transition models. This is the benchmark's own finding about our method, and it is why Section 4.1 reports medians. UG-MTM's RD in MiniGrid at maximum distance spans 17.7 to 8135 over ten seeds. Reproducing the extreme seed and instrumenting the KL term by term shows the cause is not a diverging rollout: the KL is already

Family	Level	Fine-tuning	Replay	EWC	Prog. Nets	UG-MTM
MiniGrid	min	-1.00	-1.46	-1.29	-0.76	-557
	med	-0.65	-1.93	-0.57	-1.17	-526
	max	-1.06	-2.13	-0.44	-0.15	-1642
Gymnasium	min	1.83	0.33	1.94	1.80	-4.33
	med	1.92	0.12	2.01	1.81	-14.49
	max	-2.02	-8.52	-2.15	-1.99	-243
DMControl	min	0.12	0.05	0.12	0.19*	-0.33
	med	-1.84	-8.06	-1.85	-0.82	-287
	max	-2.36	-7.40	-2.36	-1.60	-314

Table 8: FT (forward transfer) by method and cell, in pixel space: held-out task-B reconstruction of a model trained on B from scratch minus that of the pretrained model. Positive means knowledge of task A helped.

Method	after T_2	after T_3	after T_4	Peak	Pixel ×
Fine-tuning	38.58	**250.33**	125.93	T_3 (5/5)	773
Replay	25.45	40.42	**74.83**	T_4 (4/5)	3.59
EWC	29.34	**184.28**	68.42	T_3 (5/5)	759
Prog. Nets	45.79	**142.81**	65.05	T_3 (5/5)	784
UG-MTM	16.01	24.58	**35.26**	T_4 (3/5)	1.00
Difficulty of the task trained there	2.60	47.32	32.58		

Table 9: Retention of the first task as a four-task MiniGrid sequence goes on. The three numeric columns are RD of T_1 measured against the model as it stood when T_1 finished; **bold** marks each method's peak, and the Peak column counts how many of its five seeds peak there rather than reading the median curve. *Pixel* × is held-out reconstruction of T_1 after T_4, as a factor of what that model had when T_1 finished. The last row is how hard the task trained at each stage was to fit.

80 at rollout step 0, the predicted means barely move over fifteen steps, and what explodes is the post-task-B variance, with $\sigma = 0.007$ in some latent dimensions. The term $(\mu_i - \mu_k)^2/\sigma_k^2$ does the rest. Confident and wrong is a failure mode a rollout-divergence metric will expose whenever it is unbounded above, and truncating the horizon would not hide it.

4.7 The same shape appears along the sequence, not just along the axis

Everything above runs $k = 2$. The account we have drawn from it — that forgetting follows what the new task demands rather than how far it sits from the old one — makes a prediction about longer sequences that the paired grid cannot test: forgetting of the first task should rise and fall with the difficulty of whatever is being learned at the time, rather than accumulate with the number of task switches. We ran the four MiniGrid environments as one sequence, five seeds, same protocol and same budget, and measured every earlier task at every later stage.

Table 9 reports the first task's retention. Forgetting of T_1 **peaks at the third task and recedes at the fourth**, and it does so in *all five seeds* of each of the three methods that leave the encoder unprotected. It does not do so for the two that protect it, where T_1's retention drifts slowly

and monotonically instead. That split is what makes the pattern hard to read as noise: the effect appears exactly where there is something to move and is absent exactly where there is not.

The difficulty row says why. T_3 is the hardest task in the sequence to fit (47.32, against 32.58 for T_4 and 2.60 for T_2), and the peak sits on it. This is the reading of Section 4.3 appearing on a different axis: there the ordering ran over hand-labelled distance levels, here over position within a single uninterrupted run of training, with the same environments and the same budget. A peak across levels can be attributed to how the levels were constructed. A peak across position, inside one sequence, cannot.

Two further things replicate rather than emerge. The encoder column repeats the dissociation of Section 4.5 over three task switches instead of one: fine-tuning, EWC and progressive networks end with T_1's reconstruction degraded by factors of 773, 759 and 784, while EWC's RD (68.42) is among the lowest in the table — it has accumulated three Fisher diagonals, and all three are still identically zero on the encoder. And UG-MTM's pixel factor is exactly 1.00 after every stage, for the same reason it is across the grid.

We report this as one family, one sequence and five seeds. It closes the gap between the formalism and the experiments rather than establishing a new result, and the reader should weigh it accordingly: the claim it supports is that the mechanism proposed for the $k = 2$ finding also predicts what happens at $k = 4$, which is a consistency check, not an independent confirmation from new data.

4.8 Threats to validity

Ten seeds, and five in the controls. The six discriminating cells run ten seeds, which puts the exact paired permutation test's floor at $p \approx 0.002$ and is why the method comparisons of Section 4.6 can reach conventional significance at all. The three control cells remain at five, where the floor is 0.0625: no comparison drawn from them can be significant, and none is load-bearing here. Ten is still small for an effect-size estimate, and we report d_z and win counts alongside every p rather than in place of it.

Nine cells. The rank correlations of Section 4.3 pool cells from families whose scales differ by an order of magnitude. They are the interpretation of Table 4, not an independent test of it — and the ordering of the top two predictors changed when we doubled the seeds in six of the nine cells, which is the sharpest available demonstration that $n = 9$ does not resolve them.

A shared source for predictor and outcome. Task-B difficulty and RD are computed from the same runs, so a cell whose data is intrinsically hard can raise both without the one causing the other. The correlation is therefore consistent with our reading and does not establish it; what does more of the work is the Gymnasium pair, where the two levels differ by a construction we control and the harder-perturbed one is measurably the easier task to fit. A predictor built from what the optimiser did during task B, rather than from the outcome it reached, would separate the two accounts, and we did not build one.

Six discriminating cells. Three of the nine are controls (Section 4.4).

Budget. 20 random-policy episodes per task and 5000 gradient updates. Task A is learned at this scale, which a forgetting benchmark has to show before it can claim anything was forgotten: a scaling study on MiniGrid takes held-out task-A reconstruction from 6.49 at 1000 updates to 1.61 at the 5000 used here and 1.37 at 10000, so the chosen budget is past the steep part of the curve but not at its floor, and absolute magnitudes should be read as budget-dependent. Two of the nine cells were rerun at twice the budget, and what that establishes is bounded: the direction of the axis result survives in the family where the levels are nested, and d_{trans} does not (Sections 4.2 and 4.3). The other seven cells were not rerun. The protocol is declared in the config, recorded in every result

file, and refused by the runner if two budgets would be averaged into one cell.

Sequence length. The grid runs $k = 2$. One family was additionally run at $k = 4$ (Section 4.7), which is enough to show that the account survives a longer sequence and not enough to characterise how these methods behave over the ten or twenty tasks a continual learning suite would normally use. In particular UG-MTM has four experts, so a four-task sequence exhausts its capacity exactly; anything longer would report on that limit rather than on the method.

5 Discussion

5.1 What "controlled dynamic distance" can and cannot mean

We built this benchmark around three levels of dynamic distance per family and described them as controlled. The grid says that description needs amending, and the amendment is the most portable thing here, because the same construction is standard practice: continual-learning suites routinely order task sequences by a qualitative notion of similarity and treat that ordering as an experimental factor.

What is genuinely controlled is the *construction*, and in Gymnasium it is controlled in the strongest sense available: the maximum level is the medium perturbation plus two further ones, applied to the same task pair. If the axis were ordering anything monotone, a strict superset of a perturbation could not come out below it. It does, by a wide margin (Section 4.2), and the measured distance sides with the outcome rather than with the construction.

What is not controlled, then, is the quantity the label is taken to stand for. The ordering of the labels survives contact with neither of the two things we can measure: not the forgetting the cells produce, and not the measured distance between the environments' transition models, which additionally inverts MiniGrid's minimum and medium levels. A label that neither predicts the outcome nor tracks the measured construct is a name, not a control.

The practical recommendation is narrow and cheap: **report a measured distance for every task pair, and treat level labels as construction parameters rather than as an independent variable.** d_{trans} costs one model per environment — in our grid, a 20% surcharge on the total training budget — and it is defined wherever transitions can be sampled, including families where no parameter vector exists to difference.

Our own instrument is not the finished form of that recommendation, and we would rather say so than let $\rho = 0.58$ stand as an endorsement. Two defects, both established in Section 4.3. It does not resolve orderings within a family at five seeds: the gaps between Gymnasium's levels are a third of the spread within them, and doubling the training budget swaps two of the three. And as we compute it, the two reference models are trained independently, so one is scored in the other's latent basis and part of what it reports is basis mismatch rather than a difference in dynamics (Section 3.3) — which also explains why the quantity moves when the budget does, and a distance between two environments should not.

Both have concrete fixes we did not run: a reference pair sharing one frozen encoder trained on both tasks removes the confound, and more seeds resolve the orderings or show there is nothing to resolve. The recommendation we stand behind is therefore the weaker and more durable half of it — report a measured distance and hold it to account — rather than this particular estimator. It is worth noting which way the evidence ran: it was the measured quantity that exposed its own limits under a budget change, and the labelled axis that could not, because a label has nothing to be wrong about.

5.2 Two things called "distance" are being conflated

The peak at the medium level is consistent across families, which invites a mechanism rather than a list of exceptions. The one we favour is that forgetting tracks *how much the model actually updates*, and that this is non-monotone in distance for a reason that has nothing to do with similarity: overwriting requires a task that is both different and learnable, and the two conditions pull apart at the top of the axis.

Gymnasium is the case where we can say why, because the budget rerun measured it. There the maximum level is not a task the model fails to learn — it fits task B *better* than the medium level does, at both budgets. A cheetah at triple mass, half friction and 40% gravity moves very little, and the observations it generates are correspondingly less varied. The strongest perturbation in that family produces the *easiest* task B, and the least forgetting, which is the account working rather than an exception to it: what was overwritten tracked what there was to learn, not how far the environment had been moved. Note also what this does to the axis, since a perturbation is supposed to make a task harder: the construction's own premise fails at its own top level.

If that is right, the notion of task distance used in this literature bundles two variables that come apart. One is *dissimilarity*: how far the new environment's dynamics sit from the old. The other is *learning demand*: how much the model has to change to fit the new task at the available budget. They correlate at the low end — a nearby task is easy and demands little — and they decouple at the high end, where a distant task can be one the model learns almost nothing from: because it is too hard, as we suspect in DMControl, or because it is too impoverished, as we measured in Gymnasium. Those are opposite failures of the same bundling.

Our data show the seam. Task-B difficulty is the best single predictor we have ($\rho = 0.58$), and it is best precisely because it is closer to learning demand than to dissimilarity. But a single scalar cannot capture both arms of a non-monotone relation, which is why the correlation is 0.58 and not higher, and why DMControl breaks the account: there task B gets steadily harder to fit from minimum to maximum (15.43, 34.07, 40.54) while forgetting falls after the medium level — the regime where difficulty has stopped meaning "much to learn" and started meaning "too hard to learn". Separating the two would need a predictor built from what the optimiser did during task B rather than from the outcome, and we did not run that experiment.

5.3 Forgetting is not where the metrics were looking

The second result is more actionable than the first, and it is uncomfortable for the metrics we ourselves proposed.

PF and RD are computed on latents encoded once by the post-task-A model. That is what isolates the transition component, which is the benchmark's stated purpose, and it is the right choice for that purpose: letting each model encode with its own drifting VAE would score each against its own moving coordinates rather than against a common evaluation set. The consequence is that both metrics are blind to drift in the encoder, and the blindness is not a rounding error. Fine-tuning's held-out task-A reconstruction grows by a factor of 811 in MiniGrid at minimum distance while its PF reads -5.58 — in the frozen basis, an improvement. Its PF is negative in seven of nine cells.

The lesson generalises past our particular metrics: **any latent-space forgetting metric evaluated on frozen representations measures the component it was pointed at and reports nothing about the representation itself**, and in an end-to-end world model the representation is where most of the damage is. A benchmark that reports only such metrics will report that a fine-tuned model did not forget. Recording a representation-space quantity alongside them costs

one forward pass per evaluation and turns a silent scope limitation into a reported number.

EWC makes the same point structurally, and it is the cleanest result in the paper because it was derived before it was measured. EWC's Fisher information is defined over $\log P(z' \mid z, a)$; the encoder parameters do not appear in that term, so their Fisher entries are *exactly* zero — not small, zero — and the quadratic penalty cannot constrain them even in principle. The prediction has two halves and both hold: EWC preserves the transition component almost perfectly ($|\mathrm{PF}| \leq 0.07$ in seven of nine cells) while its encoder degrades by a factor of 800, indistinguishable from the unregularised baseline's 811. The Fisher is also exactly zero on `gru.weight_hh`, because the Fisher set consists of single transitions started from $h = 0$, so the recurrent pathway is unprotected as well.

This is worth stating as a general caution about regularisation-based methods: **the protected set is exactly the support of whatever likelihood the Fisher is estimated from**, and in modular architectures that support is usually narrower than the module list suggests. The failure is invisible to any metric sharing the same restriction — which, in our first formulation, was every metric we reported.

5.4 What the three mitigation families actually buy

Read across both scales, the three standard strategies separate cleanly, and none of them dominates.

Replay is the only method that keeps both components: its task-A reconstruction never grows by more than 31% and improves outright in DMControl, and it has the lower RD in four of the six cells that produce forgetting, unanimously across ten seeds in each. It pays for this where it works hardest. In the three non-MiniGrid cells where it protects the encoder, its forward transfer is the worst of the four RSSM methods by a factor of three or more (-8.52, -8.06 and -7.40, against next-worst values of -2.15, -1.85 and -2.36). Part of that is an artefact we must declare: replay trains on A and B during the task-B phase, so half its gradient steps go to task-A data and its FT confounds transfer with a halved effective budget on the new task. The rest is the expected price of rehearsal, and it is a price the metric can see only because FT is measured against a model trained on task B from scratch.

Regularisation protects what its penalty covers and nothing else, as above. Its appeal — no stored data, no growing parameter count — is intact; what our grid shows is that its guarantees are component-scoped in a way that a system-level evaluation cannot detect.

Structural isolation, including our own method, buys preservation by declining to learn. UG-MTM's task-A reconstruction is unchanged to the bit in all nine cells, because its encoder is frozen; its forward transfer is -557, -526 and -1642 in MiniGrid, where no other method leaves $[-8.52, +2.01]$ anywhere in the grid. The honest summary is that it does not forget because it does not learn. The cost nearly vanishes when the two tasks look alike (-4.33 in Gymnasium at minimum distance), which locates a regime where freezing is affordable rather than showing that the mechanism works. We report this because the benchmark produced it; a suite whose author's method came out ahead on the author's own benchmark would be worth less.

Freezing has a second consequence we did not anticipate and would not have found without the seed-level view. UG-MTM's post-task-B transition models become *overconfident*: in the extreme seed of MiniGrid's maximum-distance cell, some latent dimensions collapse to $\sigma = 0.007$, the rollout KL is already 80 at step 0, and RD reaches 4364 on that seed against a median of 576. Confidence without accuracy is a failure mode that an unbounded divergence will surface and a bounded one will hide, and it is invisible to any aggregate reported as a mean.

5.5 Lessons for building this kind of benchmark

Three of our design decisions were forced by measurements rather than chosen in advance, and we think they generalise.

Report medians when the metric is unbounded. RD is a KL divergence, so methods that produce overconfident models get heavy right tails by construction. This is not rare in our grid: 17 of 180 forgetting cells are right-skewed at five seeds. Means over five seeds describe those cells badly, and the tail is itself a result about the method rather than noise to be smoothed, so we report medians with the full seed range and mark the skewed cells.

Declare the cells that measure nothing. Three of our nine cells produce no forgetting in any method. That is the correct behaviour for the bottom of a distance axis and evidence that the axis reaches down far enough, but it also means those cells discriminate between methods on nothing, and reporting them without saying so inflates the apparent size of the grid by half. Our effective grid is six cells.

Aggregate with an explicit inclusion rule. The family-level numbers in Section 4.2 exclude UG-MTM, whose frozen encoder puts its RD two orders of magnitude below the others in DMControl. That exclusion changes the headline numbers, so it belongs in the paper and in the code that produces the tables rather than in an analysis script nobody sees.

5.6 Limitations, and what would change these conclusions

Statistical resolution was the binding constraint, and we spent compute on it rather than argument. The six cells that discriminate between methods run ten seeds, which lowers the exact paired test's floor from 0.0625 to about 0.002 and is why the comparisons in Section 5.4 can be significant at all. It also changed one of them: at five seeds replay looked better than fine-tuning in five of six forgetting cells, and at ten the sixth turns out to be a wash ($p = 0.68$) rather than a win. That is the resolution doing its job, and it is a reminder that a five-seed direction is a direction, not a result. Ten is still not many, and the three control cells remain at five; more seeds is still the cheapest improvement available.

The grid is nine cells of which six discriminate, one task switch rather than a sequence, and one architecture family: every method here is a variation on the same recurrent latent world model, so the results speak to that class and not to world models in general. Our own measured distance inherits a confound we declare but did not remove — it scores one environment's transition model in the other's latent basis — which bounds how much weight the d_{trans} half of Section 4.3 can carry. The budget — 20 random-policy episodes per task and 5000 gradient updates — is small enough that absolute magnitudes are budget-dependent, although task A is demonstrably learned at this scale, which is the precondition a forgetting benchmark has to establish before any of its numbers mean anything.

Two specific results would be worth attacking. The peak at the medium level is the finding we are most confident in, since it repeats in three of three families and survives changes of aggregation, metric and threshold; it would be substantially strengthened by a family in which the levels are constructed to be monotone in d_{trans} and the peak persists anyway. We did not do this here, and we note that it has to be done carefully: choosing levels so that the measured distance comes out monotone, and then reporting that forgetting follows it, would be circular. The prediction that distinguishes our account from the design's is testable and cheap — forgetting should fall again at distances large enough that task B stops being learnable at the given budget.

The second is policy-level impact. An earlier formulation of this benchmark announced a policy-impact metric alongside the three we report; it was never implemented, because measuring it means

training a controller inside the model's imagination and evaluating it in the real environment, and we withdraw it rather than report a placeholder. It remains the most interesting gap: everything here measures degradation of the model, and whether the ordering we find survives the translation into control performance is an open question that this suite, as it stands, cannot answer.

5.7 On the previous version of these results

An earlier version of this benchmark reported five findings, at significance levels a five-seed design cannot produce. None of them survives re-measurement, and the causes were defects in the instrument rather than in the analysis: a collapsed VAE posterior that fed the transition model a constant latent, an evaluation set of Gaussian noise, a next-state objective scored against the wrong target, a mis-specified Gaussian KL that inflated RD roughly twelvefold, and unseeded environments that made five seeds five draws of different data rather than five replicates.

We report this deliberately. The strongest of those findings — that replay is insufficient in low-capacity settings — was the one that reversed, and it was also the most counterintuitive, which is the pattern to expect when a measurement is broken: an artefact is more likely to look surprising than a real effect is. The infrastructure that replaced it is the substrate the rest of this paper rests on. Runs reproduce bit for bit across processes and across days: a cell re-executed eight days after the original run, to diagnose the heavy tail of Section 5.4, returned RD = 4364.4502 against 4364.4502 stored and PF = 39.9796 against 39.9796. Every result records the protocol it ran under and the task pair it came from, and the runner refuses to combine results produced under different ones. Those properties are unglamorous and they are what makes the negative result in Section 4.2 worth reporting at all.

6 Conclusion

We set out to build the missing instrument — a benchmark for catastrophic forgetting in the transition component of a world model, rather than in a policy or in an agent as a whole — and the instrument's most useful output was a correction to two things the field does by default.

The first is a design habit. Continual learning experiments are routinely built around an ordinal notion of how far apart the tasks in a sequence are, and that ordering is then treated as an experimental factor. Ours does not order forgetting: RD peaks at the medium level in all three families, the labelled level carries no rank information across the nine cells, and the cleanest case needs no comparison between families at all — one family's maximum level is its medium perturbation plus two more, on the same task pair, and it forgets less. That survives a doubling of the training budget, a doubling of the seed count, and a four-task sequence, and in the one case we could interrogate directly the obvious explanation turns out to be backwards: the more heavily perturbed task is the *easier* one to fit. What we can say is that forgetting tracked what the new task demanded of the model rather than how far it sat from the old one. What we cannot yet say is which of the two measurable quantities standing in for that demand should be reported, since they swapped rank when we added seeds.

The second is a measurement habit. Isolating a component means holding a representation fixed, and a metric defined on a fixed representation cannot see that representation move. Ours cannot: fine-tuning's held-out reconstruction of the first task degrades by a factor of 811 while its prediction fidelity records an improvement. EWC turns that from an artefact into a structural prediction — its Fisher information is identically zero on every encoder parameter, so it protects the transition component almost exactly and nothing else — and both halves of the prediction were checkable

before the experiment ran. A component-level benchmark owes its readers both scales, and ours reports them side by side for that reason.

Neither result is about which method wins, and neither is comfortable for the method we proposed: UG-MTM does not forget because it freezes its encoder, and pays for it in transfer. We report it as one of five characterised methods.

The limits are the ones stated in Section 4.8 and we do not think they should be read past. This is one architecture family at a small budget on data from a random policy, and nothing here establishes what happens at the scale world models are actually trained at. The natural next step is not a better aggregate or another mitigation method but the same measurements on a model somebody would deploy — and the second is a distance between environments that survives contact with its own instrument, which ours does not.

Code, configurations, the plotting and table-generating scripts, and all 375 result files are released.

References

[1] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[2] Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.

[3] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning (ICML)*, 2016.

[4] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[5] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019.

[6] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

[7] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.

[8] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[9] Samuel Kessler, Mateusz Ostaszewski, Michał P. Bortkiewicz, Mateusz Żarski, Maciej Wołczyk, Jack Parker-Holder, Stephen J. Roberts, and Piotr Miłoś. The effectiveness of world models for continual reinforcement learning. In *Conference on Lifelong Learning Agents (CoLLAs)*, volume 232 of *PMLR*, pages 184–204, 2023.

[10] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[11] Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[12] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[13] Roger Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97(2):285–308, 1990.

[14] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[15] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

[16] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.

[17] Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm_control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020.

[18] Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. Continual learning and catastrophic forgetting. *arXiv preprint arXiv:2403.05175*, 2024.

[19] Maciej Wołczyk, Michał Zając, Razvan Pascanu, Łukasz Kuciński, and Piotr Miłoś. Continual world: A robotic benchmark for continual reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[20] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning (ICML)*, 2017.

A Was there anything to forget?

A forgetting benchmark that has not shown its models learned the first task is reporting the distance between two bad models. Table 10 is that check, for every cell. Training on task A moves the one-step NLL on D_A from the high thirties or low forties to between 10.68 and 15.65 in every cell, and the held-out reconstruction reaches values that differ by two orders of magnitude between the visually simplest cells and the hardest — which is the task-difficulty signal Section 4.3 uses, seen at the first task instead of the second.

Family	Level	Held-out recon.	NLL at init	NLL after A
MiniGrid	min	0.48	37.19	14.89
	med	0.84	38.65	13.78
	max	42.58	37.45	15.65
Gymnasium	min	26.49	41.30	12.42
	med	26.15	41.30	12.42
	max	26.82	41.50	12.59
DMControl	min	14.74	29.61	10.68
	med	14.54	29.63	11.73
	max	14.54	29.63	11.73

Table 10: Task-A quality per cell, measured before any task switch and pooled over methods — at that point in the protocol every method has trained on task A and nothing else. *Held-out recon.* is squared error summed over the $3 \times 64 \times 64$ frame on episodes never trained on. The two NLL columns are the transition component's one-step negative log-likelihood on D_A at initialisation and after training, so their difference is what training bought.

The one cell that deserves comment is MiniGrid at maximum distance, whose held-out reconstruction of task A is 42.58 where the other two MiniGrid cells are below 1. Its task A is FourRooms, which is the hardest environment in that family to fit; the model still improves its NLL on D_A by more than a factor of two, so there is something learned there to lose, but the pixel-scale comparisons in that row are between larger numbers than elsewhere.

B Where the budget sits on the convergence curve

The protocol trains 5000 gradient updates per task. That number was chosen from a scaling probe on MiniGrid rather than by default: a single run evaluated at 1000, 2000, 5000 and 10000 updates takes held-out task-A reconstruction from 6.49 to 3.01, 1.61 and 1.37. The step from 1000 to 5000 removes three quarters of the error; the step from 5000 to 10000 removes a further 15%.

We report that as locating the budget past the steep part of the curve and short of its floor, which is what a forgetting benchmark needs — a model that has not converged has less to lose, and one that has fully converged is not the regime anybody trains in either. It does not license reading the absolute magnitudes as budget-independent, and Section 4.2 is the one place where we tested the direction of a result against a change of budget rather than assuming it.

These four numbers come from a separate probe rather than from the grid, which is why they do not appear in a generated table.

C Reproducibility

Every result in this paper is produced by two scripts that train nothing: one prints the console summary and one emits the paper's tables in LaTeX by importing the first. Every table in the paper carries a header saying it was generated. This is not tidiness — the previous version of this work transcribed its tables by hand and they drifted from the runs they claimed to describe, and three different sets of protocol values circulated for the same experiment.

Runs are reproducible from their seed, and the part that took longest to get right was not the

framework's global RNG but each environment's own: Gymnasium's action space and dm_control's `RandomState` are not reached by seeding NumPy, and until they were seeded the five seeds of a cell trained on five different datasets and were not replicates. All instrumentation is wrapped so that measuring cannot perturb a run, which is load-bearing because UG-MTM's gate keeps MC-dropout active at evaluation time by design and so consumes the random stream. We reproduced a cell bit for bit eight days after it first ran, in a different process, after the runner had been rewritten in between.

Each result file records the protocol block it was produced under and the task pair it was run on. The runner refuses to reuse a cached result that disagrees with either, the loader refuses to aggregate a directory whose runs disagree, and the table exporter refuses to emit from one. Those three checks share one definition of what makes two results comparable.

D Where the measured distance fails

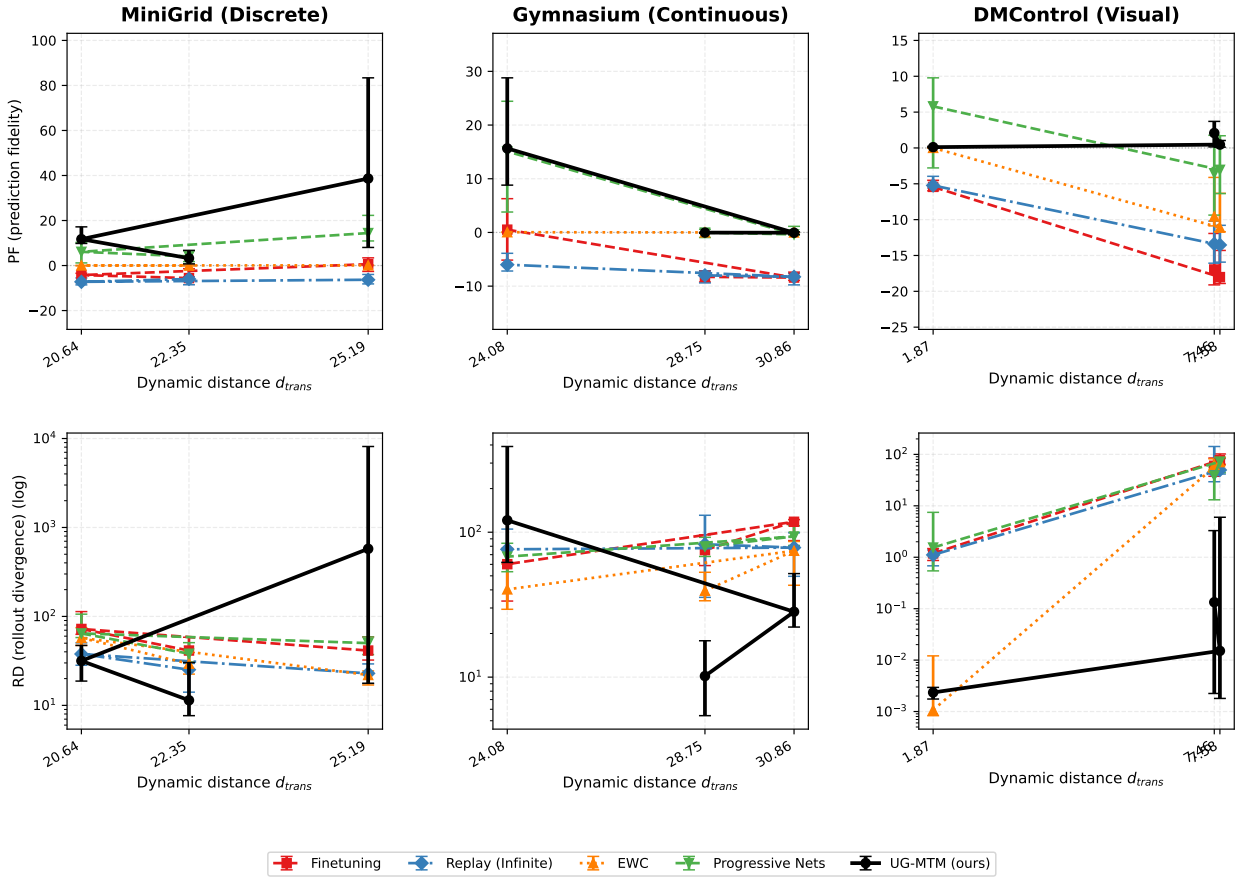

Figure 2: The same runs as Figure 1, plotted against the measured distance d_{trans} instead of the labelled level. Compare the x-axis ticks: within each family the three levels sit almost on top of one another, and in DMControl two of the three labels collide. This is Section 4.3's point made visually — the measured distance separates families cleanly and does not resolve the levels inside one.

Figure 2 is the figure this paper would have led with under its earlier framing, when d_{trans} was believed to be the better axis. We include it because what it shows now is the opposite of what it

was drawn for, and because the failure is easier to see than to state: the cells do not spread out along the measured axis, and re-sorting them by it is what makes the peak of Figure 1 invisible.